ДОДАТОК Г

Кросплатформна клієнт-серверна система агрегації та обробки медико-біологічних даних з використанням хмарних технологій

Текст програмного коду

ІАЛЦ.467200.007 Д4

Аркушів 21

Київ 2026 р

```
-- supabase/migrations/20260423132703_initial_schema.sql
-- ============================================================================
-- Initial schema: core domain for a baby health tracker
-- ============================================================================

create extension if not exists "pgcrypto";

create type caregiver_role as enum (
  'owner', 'co_parent', 'caregiver', 'pediatrician');

create type feeding_kind as enum (
  'breast_left', 'breast_right',
  'bottle_breast_milk', 'bottle_formula', 'solid');

create type diaper_kind as enum ('wet', 'dirty', 'mixed', 'dry');

create type measurement_kind as enum (
  'weight', 'height', 'head_circumference');

-- Profiles - one row per auth user
create table public.profiles (
  id          uuid primary key references auth.users(id) on delete cascade,
  full_name   text,
  avatar_url  text,
  locale      text default 'uk',
  created_at  timestamptz not null default now(),
  updated_at  timestamptz not null default now()
);

create or replace function public.handle_new_user()
returns trigger language plpgsql security definer
set search_path = public as $$
begin
  insert into public.profiles (id, full_name)
  values (new.id, coalesce(new.raw_user_meta_data->>'full_name', ''));
  return new;
end;
$$;

create trigger on_auth_user_created
  after insert on auth.users
  for each row execute function public.handle_new_user();
```

					ІАЛЦ.467200.007 Д4		
Зм.	Лист	№ докум.	Підп.	Дата			
Розробив		Льоскін І. В.			Кросплатформна клієнт-серверна система агрегації та обробки медико-біологічних даних з використанням хмарних технологій Текст програмного коду	Лит.	Аркуш 1 / Аркушів 21
Перевірив		Кобилюк А. Г.					
Рецен.		Сергієнко П.А.				НТУУ "КПІ ім. Ігоря Сікорського", ФІОТ ІО-25	
Н. контр.		Нікольський С.С.					
Затвердив		Волокита А. М.					

```
-- Children
create table public.children (
  id            uuid primary key default gen_random_uuid(),
  full_name     text not null,
  date_of_birth date not null,
  sex           text check (sex in ('male', 'female', 'unspecified'))
                  default 'unspecified',
  avatar_url    text,
  notes         text,
  created_at    timestamptz not null default now(),
  updated_at    timestamptz not null default now()
);

-- Caregivers - M:N between profiles and children, with roles

create table public.caregivers (
  child_id    uuid not null references public.children(id) on delete cascade,
  profile_id  uuid not null references public.profiles(id) on delete cascade,
  role        caregiver_role not null default 'co_parent',
  created_at  timestamptz not null default now(),
  primary key (child_id, profile_id)
);

create index caregivers_profile_idx on public.caregivers(profile_id);

create or replace function public.is_caregiver(p_child uuid)
returns boolean language sql stable security definer
set search_path = public as $$
  select exists (
    select 1 from public.caregivers
    where child_id = p_child and profile_id = auth.uid()
  );
$$;

-- Feedings
create table public.feedings (
  id          uuid primary key default gen_random_uuid(),
  child_id    uuid not null references public.children(id) on delete cascade,
  kind        feeding_kind not null,
  started_at  timestamptz not null,
  ended_at    timestamptz,
  amount_ml   numeric(6,1),
  notes       text,
  created_by  uuid references public.profiles(id),
  created_at  timestamptz not null default now()
);
```

```
create index feedings_child_started_idx
  on public.feedings(child_id, started_at desc);

-- Sleeps
create table public.sleeps (
  id          uuid primary key default gen_random_uuid(),
  child_id    uuid not null references public.children(id) on delete cascade,
  started_at  timestamptz not null,
  ended_at    timestamptz,
  notes       text,
  created_by  uuid references public.profiles(id),
  created_at  timestamptz not null default now()
);
create index sleeps_child_started_idx
  on public.sleeps(child_id, started_at desc);

-- Diapers
create table public.diapers (
  id          uuid primary key default gen_random_uuid(),
  child_id    uuid not null references public.children(id) on delete cascade,
  kind        diaper_kind not null,
  occurred_at timestamptz not null default now(),
  notes       text,
  created_by  uuid references public.profiles(id),
  created_at  timestamptz not null default now()
);
create index diapers_child_occurred_idx
  on public.diapers(child_id, occurred_at desc);

-- Growth measurements
create table public.growth_measurements (
  id          uuid primary key default gen_random_uuid(),
  child_id    uuid not null references public.children(id) on delete cascade,
  kind        measurement_kind not null,
  value       numeric(7,2) not null,
  unit        text not null,
  measured_at timestamptz not null default now(),
  notes       text,
  created_by  uuid references public.profiles(id),
  created_at  timestamptz not null default now()
);
create index growth_child_measured_idx
  on public.growth_measurements(child_id, kind, measured_at desc);

-- Milestone templates (seeded once)
create table public.milestone_templates (
```

```
  id                      uuid primary key default gen_random_uuid(),
  code                    text unique not null,
  title                   text not null,
  description             text,
  category                text not null,
  age_min_months          int not null,
  age_max_months          int not null
);

-- Milestones (per-child achievements)
create table public.milestones (
  id          uuid primary key default gen_random_uuid(),
  child_id    uuid not null references public.children(id) on delete cascade,
  template_id uuid references public.milestone_templates(id),
  custom_title text,
  achieved_at date,
  notes       text,
  created_by  uuid references public.profiles(id),
  created_at  timestamptz not null default now(),
  check (template_id is not null or custom_title is not null)
);
create index milestones_child_idx on public.milestones(child_id);

-- Vaccinations
create table public.vaccinations (
  id              uuid primary key default gen_random_uuid(),
  child_id        uuid not null references public.children(id) on delete cascade,
  vaccine_code    text not null,
  vaccine_name    text not null,
  scheduled_for   date,
  administered_at date,
  dose_number     int,
  notes           text,
  created_by      uuid references public.profiles(id),
  created_at      timestamptz not null default now()
);
create index vaccinations_child_idx on public.vaccinations(child_id);

-- Row Level Security
alter table public.profiles           enable row level security;
alter table public.children           enable row level security;
alter table public.caregivers         enable row level security;
alter table public.feedings           enable row level security;
alter table public.sleeps             enable row level security;
alter table public.diapers            enable row level security;
alter table public.growth_measurements enable row level security;
```

```
alter table public.milestone_templates enable row level security;
alter table public.milestones         enable row level security;
alter table public.vaccinations       enable row level security;

create policy "profiles_self_select" on public.profiles
  for select using (id = auth.uid());
create policy "profiles_self_update" on public.profiles
  for update using (id = auth.uid());

create policy "children_caregiver_select" on public.children
  for select using (public.is_caregiver(id));
create policy "children_authenticated_insert" on public.children
  for insert with check (auth.uid() is not null);
create policy "children_caregiver_update" on public.children
  for update using (public.is_caregiver(id));

create policy "caregivers_caregiver_select" on public.caregivers
  for select using (public.is_caregiver(child_id));

create policy "feedings_caregiver_select" on public.feedings
  for select using (public.is_caregiver(child_id));
create policy "feedings_caregiver_insert" on public.feedings
  for insert with check (public.is_caregiver(child_id));
create policy "feedings_caregiver_update" on public.feedings
  for update using (public.is_caregiver(child_id));
create policy "feedings_caregiver_delete" on public.feedings
  for delete using (public.is_caregiver(child_id));

-- supabase/migrations/20260425132539_child_owner_trigger.sql
-- Auto-add the creator as the owner caregiver of every new child.
-- Lets the client do a single INSERT into children; Postgres handles
-- the caregivers row atomically inside a SECURITY DEFINER trigger.

create or replace function public.add_child_owner()
returns trigger language plpgsql security definer
set search_path = public as $$
begin
  if auth.uid() is not null then
    insert into public.caregivers (child_id, profile_id, role)
    values (new.id, auth.uid(), 'owner')
    on conflict (child_id, profile_id) do nothing;
  end if;
  return new;
end;
$$;

drop trigger if exists on_child_created on public.children;
```

```
create trigger on_child_created
  after insert on public.children
  for each row execute function public.add_child_owner();

-- supabase/migrations/20260513200000_articles_pgvector.sql
-- Enable pgvector for semantic search.
-- gte-small produces 384-dimensional embeddings via Supabase AI.
create extension if not exists vector;

create table public.articles (
  id             uuid primary key default gen_random_uuid(),
  title          text not null,
  body           text not null,
  category       text not null,
  age_min_months int  not null default 0,
  age_max_months int  not null default 36,
  embedding      vector(384),
  created_at     timestamptz not null default now()
);

-- IVFFlat index for approximate nearest-neighbour search.
-- 20 lists is appropriate for <1 000 rows; increase for larger corpora.
create index articles_embedding_idx on public.articles
  using ivfflat (embedding vector_cosine_ops) with (lists = 20);

alter table public.articles enable row level security;
create policy "articles_authenticated_read" on public.articles
  for select using (auth.uid() is not null);

-- Semantic search: returns rows ranked by cosine similarity to the
-- query embedding, optionally filtered by child age.
create or replace function public.match_articles(
  query_embedding vector(384),
  match_count     int     default 3,
  p_age_months    int     default null
)
returns table (
  id         uuid,
  title      text,
  body       text,
  category   text,
  similarity float
)
language sql stable
as $$
  select id, title, body, category,
    1 - (embedding <=> query_embedding) as similarity
```

```
  from public.articles
  where embedding is not null
    and (
      p_age_months is null
      or (age_min_months <= p_age_months and age_max_months >= p_age_months)
    )
  order by embedding <=> query_embedding
  limit match_count;
$$;

// app/types/domain.ts
/**
 * Domain types - re-exports derived from the auto-generated Supabase schema.
 * App code should import from here instead of `lib/database.types` directly.
 */
import type { Database } from '@/lib/database.types';

type Tables = Database['public']['Tables'];
type Enums = Database['public']['Enums'];

export type Child = Tables['children']['Row'];
export type ChildInsert = Tables['children']['Insert'];
export type ChildUpdate = Tables['children']['Update'];
export type Sex = 'male' | 'female' | 'unspecified';

export type Caregiver = Tables['caregivers']['Row'];
export type CaregiverRole = Enums['caregiver_role'];

export type Feeding = Tables['feedings']['Row'];
export type FeedingInsert = Tables['feedings']['Insert'];
export type FeedingKind = Enums['feeding_kind'];

export type Sleep = Tables['sleeps']['Row'];
export type SleepInsert = Tables['sleeps']['Insert'];

export type Diaper = Tables['diapers']['Row'];
export type DiaperInsert = Tables['diapers']['Insert'];
export type DiaperKind = Enums['diaper_kind'];

export type GrowthMeasurement = Tables['growth_measurements']['Row'];
export type GrowthMeasurementInsert = Tables['growth_measurements']['Insert'];
export type MeasurementKind = Enums['measurement_kind'];

export type Milestone = Tables['milestones']['Row'];
export type MilestoneTemplate = Tables['milestone_templates']['Row'];

export type Vaccination = Tables['vaccinations']['Row'];
```

```ts
export type VaccinationInsert = Tables['vaccinations']['Insert'];
export type VaccinationTemplate = Tables['vaccination_templates']['Row'];

// app/lib/supabase.ts
import 'react-native-url-polyfill/auto';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { createClient } from '@supabase/supabase-js';
import { AppState } from 'react-native';
import type { Database } from './database.types';
import { env } from './env';

export const supabase = createClient<Database>(env.supabaseUrl, env.supabaseAnonKey, {
  auth: {
    storage: AsyncStorage,
    autoRefreshToken: true,
    persistSession: true,
    detectSessionInUrl: false,
  },
});

AppState.addEventListener('change', (state) => {
  if (state === 'active') {
    supabase.auth.startAutoRefresh();
  } else {
    supabase.auth.stopAutoRefresh();
  }
});

/**
 * Reads the persisted session and returns the current user's id, or `null`
 * when no session exists. Used by every domain mutation to stamp
 * `created_by` on inserts.
 */
export async function getCurrentUserId(): Promise<string | null> {
  const { data } = await supabase.auth.getSession();
  return data.session?.user.id ?? null;
}

// app/features/measurements/who/index.ts
import { differenceInMonths, parseISO } from 'date-fns';

import {
  WHO_STANDARDS,
  type WhoMetric,
  type WhoRow,
  type WhoSex,
} from './standards';
```

```
import type { GrowthMeasurement, MeasurementKind, Sex } from '@/types/domain';

const KIND_TO_METRIC: Record<MeasurementKind, WhoMetric> = {
  weight: 'weight',
  height: 'height',
  head_circumference: 'head_circumference',
};

function whoSex(sex: Sex | string | null | undefined): WhoSex {
  return sex === 'female' ? 'female' : 'male';
}

export type Bands = {
  p3: number;
  p15: number;
  p50: number;
  p85: number;
  p97: number;
};

const linterp = (a: number, b: number, t: number) => a + (b - a) * t;

function lookup(rows: readonly WhoRow[], ageMonths: number): Bands | null {
  if (ageMonths < rows[0][0] || ageMonths > rows[rows.length - 1][0])
    return null;
  const upperIdx = rows.findIndex((r) => r[0] >= ageMonths);
  if (upperIdx === -1) return null;
  const upper = rows[upperIdx];
  if (upper[0] === ageMonths || upperIdx === 0) {
    return {
      p3: upper[1], p15: upper[2], p50: upper[3],
      p85: upper[4], p97: upper[5],
    };
  }
  const lower = rows[upperIdx - 1];
  const t = (ageMonths - lower[0]) / (upper[0] - lower[0]);
  return {
    p3:  linterp(lower[1], upper[1], t),
    p15: linterp(lower[2], upper[2], t),
    p50: linterp(lower[3], upper[3], t),
    p85: linterp(lower[4], upper[4], t),
    p97: linterp(lower[5], upper[5], t),
  };
}

export function whoBands(
```

```
  sex: Sex | string | null | undefined,
  kind: MeasurementKind,
  ageMonths: number,
): Bands | null {
  return lookup(
    WHO_STANDARDS[whoSex(sex)][KIND_TO_METRIC[kind]],
    ageMonths,
  );
}

export function ageMonthsAt(dobIso: string, atIso: string): number {
  return Math.max(differenceInMonths(parseISO(atIso), parseISO(dobIso)), 0);
}

export type PercentileBand =
  | 'below_p3' | 'p3_p15' | 'p15_p85' | 'p85_p97' | 'above_p97';

export function classifyValue(value: number, bands: Bands): PercentileBand {
  if (value < bands.p3)  return 'below_p3';
  if (value < bands.p15) return 'p3_p15';
  if (value <= bands.p85) return 'p15_p85';
  if (value <= bands.p97) return 'p85_p97';
  return 'above_p97';
}

export function classifyMeasurement(
  measurement: GrowthMeasurement,
  dobIso: string,
  sex: Sex | string | null | undefined,
): { bands: Bands; band: PercentileBand } | null {
  const months = ageMonthsAt(dobIso, measurement.measured_at);
  const bands = whoBands(sex, measurement.kind, months);
  if (!bands) return null;
  return { bands, band: classifyValue(measurement.value, bands) };
}

// app/hooks/use-sleep-pattern.ts
import { useMemo } from 'react';
import { addDays, format, parseISO, startOfDay } from 'date-fns';

import { useSleepsInRange } from '@/features/sleeps/queries';
import { useLastDaysRange } from '@/hooks/use-last-days-range';
import type { Sleep } from '@/types/domain';

/** A single block within one day, hours expressed as floats in [0, 24]. */
export type SleepBlock = { startHour: number; endHour: number };
```

```
export type SleepPatternDay = {
  /** ISO day key (yyyy-MM-dd). */
  key: string;
  /** Calendar date for axis labelling. */
  date: Date;
  blocks: SleepBlock[];
};

const HOURS_IN_DAY = 24;
const MS_IN_HOUR = 60 * 60 * 1000;

function splitSleepAtMidnight(
  sleep: Sleep,
  windowStart: Date,
  windowEnd: Date,
): { dayKey: string; block: SleepBlock }[] {
  const start = parseISO(sleep.started_at);
  const end = sleep.ended_at ? parseISO(sleep.ended_at) : new Date();

  // Clip the sleep to the visible window so a sleep that started weeks ago
  // and only finished today still produces just one block.
  const clipStart = start < windowStart ? windowStart : start;
  const clipEnd = end > windowEnd ? windowEnd : end;
  if (clipEnd <= clipStart) return [];

  const out: { dayKey: string; block: SleepBlock }[] = [];
  let cursor = clipStart;
  while (cursor < clipEnd) {
    const dayStart = startOfDay(cursor);
    const nextDayStart = addDays(dayStart, 1);
    const segmentEnd = clipEnd < nextDayStart ? clipEnd : nextDayStart;
    out.push({
      dayKey: format(dayStart, 'yyyy-MM-dd'),
      block: {
        startHour: (cursor.getTime() - dayStart.getTime()) / MS_IN_HOUR,
        endHour: (segmentEnd.getTime() - dayStart.getTime()) / MS_IN_HOUR,
      },
    });
    cursor = segmentEnd;
  }
  return out;
}

/**
 * Buckets the last `days` days of sleep entries for the active child into a
 * per-day list of intra-day blocks suitable for a 24-hour pattern chart.
```

```
 * Sleeps that span midnight are split into two adjacent blocks; an
 * unfinished sleep is rendered up to the current moment.
 */
export function useSleepPattern(childId: string | null, days: number): SleepPatternDay[] {
  const { from, to } = useLastDaysRange(days);
  const { data: sleeps = [] } = useSleepsInRange(childId, from, to);

  return useMemo(() => {
    const today = startOfDay(new Date());
    const buckets: SleepPatternDay[] = [];
    for (let i = days - 1; i >= 0; i--) {
      const date = addDays(today, -i);
      buckets.push({ key: format(date, 'yyyy-MM-dd'), date, blocks: [] });
    }
    const byKey = new Map(buckets.map((b) => [b.key, b]));

    const windowStart = buckets[0].date;
    const windowEnd = addDays(today, 1);

    for (const sleep of sleeps) {
      for (const { dayKey, block } of splitSleepAtMidnight(sleep, windowStart, windowEnd)) {
        const bucket = byKey.get(dayKey);
        if (bucket) bucket.blocks.push(block);
      }
    }
    return buckets;
  }, [sleeps, days]);
}

export const SLEEP_PATTERN_HOURS = HOURS_IN_DAY;

// app/lib/realtime.ts
import { useEffect } from 'react';
import { useQueryClient } from '@tanstack/react-query';

import { supabase } from './supabase';

const TABLE_TO_KEY: Record<string, string> = {
  feedings:           'feedings',
  sleeps:             'sleeps',
  diapers:            'diapers',
  growth_measurements: 'measurements',
  milestones:         'milestones',
  vaccinations:       'vaccinations',
};

export function useChildRealtime(childId: string | null) {
```

```
  const qc = useQueryClient();

  useEffect(() => {
    if (!childId) return;

    const channel = supabase.channel(`child:${childId}`);

    for (const table of Object.keys(TABLE_TO_KEY)) {
      channel.on(
        'postgres_changes',
        {
          event: '*',
          schema: 'public',
          table,
          filter: `child_id=eq.${childId}`,
        },
        () => {
          qc.invalidateQueries({
            queryKey: [TABLE_TO_KEY[table], childId],
          });
        },
      );
    }

    channel.subscribe();

    return () => {
      supabase.removeChannel(channel);
    };
  }, [childId, qc]);
}

// app/features/feedings/queries.ts
import { useMutation, useQuery, useQueryClient } from '@tanstack/react-query';
import { startOfDay, endOfDay, format } from 'date-fns';

import { getCurrentUserId, supabase } from '@/lib/supabase';
import type { Feeding, FeedingInsert, FeedingKind } from '@/types/domain';

export const feedingsKey = (childId: string) =>
  ['feedings', childId] as const;
export const feedingsForDayKey = (childId: string, day: string) =>
  ['feedings', childId, 'day', day] as const;
export const feedingsInRangeKey =
  (childId: string, from: string, to: string) =>
    ['feedings', childId, 'range', from, to] as const;
export const activeFeedingKey = (childId: string) =>
```

```
  ['feedings', childId, 'active'] as const;

export function useFeedingsInRange(
  childId: string | null,
  from: Date,
  to: Date,
) {
  const fromKey = format(from, 'yyyy-MM-dd');
  const toKey   = format(to,   'yyyy-MM-dd');
  return useQuery({
    queryKey: childId
      ? feedingsInRangeKey(childId, fromKey, toKey)
      : ['feedings', 'noop-range'],
    enabled: Boolean(childId),
    queryFn: async (): Promise<Feeding[]> => {
      if (!childId) return [];
      const { data, error } = await supabase
        .from('feedings')
        .select('*')
        .eq('child_id', childId)
        .gte('started_at', startOfDay(from).toISOString())
        .lte('started_at', endOfDay(to).toISOString())
        .order('started_at', { ascending: true });
      if (error) throw error;
      return data;
    },
  });
}

export function useFeedingsForDay(childId: string | null, day: Date) {
  const dayKey = format(day, 'yyyy-MM-dd');
  return useQuery({
    queryKey: childId
      ? feedingsForDayKey(childId, dayKey)
      : ['feedings', 'noop'],
    enabled: Boolean(childId),
    queryFn: async (): Promise<Feeding[]> => {
      if (!childId) return [];
      const { data, error } = await supabase
        .from('feedings')
        .select('*')
        .eq('child_id', childId)
        .gte('started_at', startOfDay(day).toISOString())
        .lte('started_at', endOfDay(day).toISOString())
        .order('started_at', { ascending: false });
      if (error) throw error;
```

```
      return data;
    },
  });
}

export function useActiveFeeding(childId: string | null) {
  return useQuery({
    queryKey: childId
      ? activeFeedingKey(childId)
      : ['feedings', 'noop-active'],
    enabled: Boolean(childId),
    queryFn: async (): Promise<Feeding | null> => {
      if (!childId) return null;
      const { data, error } = await supabase
        .from('feedings')
        .select('*')
        .eq('child_id', childId)
        .in('kind', ['breast_left', 'breast_right'])
        .is('ended_at', null)
        .order('started_at', { ascending: false })
        .limit(1)
        .maybeSingle();
      if (error) throw error;
      return data;
    },
  });
}

export function useStartFeeding() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async ({
      childId,
      kind,
    }: {
      childId: string;
      kind: FeedingKind;
    }): Promise<Feeding> => {
      const userId = await getCurrentUserId();
      const { data, error } = await supabase
        .from('feedings')
        .insert({
          child_id:   childId,
          kind,
          started_at: new Date().toISOString(),
          ended_at:   null,
```

```
          created_by: userId,
        })
        .select('*')
        .single();
      if (error) throw error;
      return data;
    },
    onSuccess: (feeding) => {
      qc.invalidateQueries({ queryKey: feedingsKey(feeding.child_id) });
    },
  });
}

export function useStopFeeding() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async (feedingId: string): Promise<Feeding> => {
      const { data, error } = await supabase
        .from('feedings')
        .update({ ended_at: new Date().toISOString() })
        .eq('id', feedingId)
        .select('*')
        .single();
      if (error) throw error;
      return data;
    },
    onSuccess: (feeding) => {
      qc.invalidateQueries({ queryKey: feedingsKey(feeding.child_id) });
    },
  });
}

export function useCreateFeeding() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: async (input: FeedingInsert): Promise<Feeding> => {
      const userId = await getCurrentUserId();
      const { data, error } = await supabase
        .from('feedings')
        .insert({ ...input, created_by: userId })
        .select('*')
        .single();
      if (error) throw error;
      return data;
    },
    onSuccess: (feeding) => {
```

```
      qc.invalidateQueries({ queryKey: feedingsKey(feeding.child_id) });
    },
  });
}

// app/features/auth/mutations.ts
import { useMutation } from '@tanstack/react-query';

import { supabase } from '@/lib/supabase';

export function useSignIn() {
  return useMutation({
    mutationFn: async (input: { email: string; password: string }) => {
      const { error } = await supabase.auth.signInWithPassword({
        email:    input.email.trim(),
        password: input.password,
      });
      if (error) throw error;
    },
  });
}

export function useSignUp() {
  return useMutation({
    mutationFn: async (input: {
      email:    string;
      password: string;
      fullName: string;
    }): Promise<{ needsConfirmation: boolean }> => {
      const { error, data } = await supabase.auth.signUp({
        email:    input.email.trim(),
        password: input.password,
        options:  { data: { full_name: input.fullName.trim() } },
      });
      if (error) throw error;
      return { needsConfirmation: !data.session };
    },
  });
}

export function useSignOut() {
  return useMutation({
    mutationFn: async () => {
      const { error } = await supabase.auth.signOut();
      if (error) throw error;
    },
  });
```

```
}

export function useUpdateEmail() {
  return useMutation({
    mutationFn: async (email: string) => {
      const { error } = await supabase.auth.updateUser({ email });
      if (error) throw error;
    },
  });
}

export function useUpdatePassword() {
  return useMutation({
    mutationFn: async (password: string) => {
      const { error } = await supabase.auth.updateUser({ password });
      if (error) throw error;
    },
  });
}

export function useDeleteMyAccount() {
  return useMutation({
    mutationFn: async () => {
      const { error } = await supabase.rpc('delete_my_account');
      if (error) throw error;
      await supabase.auth.signOut();
    },
  });
}

// supabase/functions/ask-assistant/index.ts
//
// RAG pipeline:
//   1. Embed user prompt via Supabase AI (gte-small, 384 dims) - free
//   2. Semantic search over articles via pgvector cosine distance
//   3. Build context from top-3 articles
//   4. Call Groq (Llama 3.3 70B) for a grounded answer - free tier
//
// Falls back to full-text ILIKE search when embeddings are not yet
// computed (i.e., right after seeding before embed-articles runs).
//
// POST /functions/v1/ask-assistant
// { "prompt": "...", "child_age_months": 6 }

import { serve } from 'https://deno.land/std@0.224.0/http/server.ts';
import { createClient } from 'https://esm.sh/@supabase/supabase-js@2';
```

```
const corsHeaders: Record<string, string> = {
  'Access-Control-Allow-Origin': '*',
  'Access-Control-Allow-Headers': 'authorization, x-client-info, apikey, content-type',
};

const json = (status: number, body: unknown) =>
  new Response(JSON.stringify(body), {
    status,
    headers: { ...corsHeaders, 'Content-Type': 'application/json' },
  });

type ArticleSource = { id: string; title: string; category: string };

serve(async (req) => {
  if (req.method === 'OPTIONS') return new Response(null, { headers: corsHeaders });
  if (req.method !== 'POST') return json(405, { error: 'method_not_allowed' });

  const auth = req.headers.get('Authorization');
  if (!auth?.startsWith('Bearer ')) return json(401, { error: 'missing_auth' });

  let body: { prompt?: string; child_age_months?: number | null } = {};
  try {
    body = await req.json();
  } catch {
    return json(400, { error: 'invalid_json' });
  }

  const prompt = body.prompt?.trim();
  if (!prompt) return json(400, { error: 'prompt_required' });

  const childAgeMonths =
    typeof body.child_age_months === 'number' ? body.child_age_months : null;

  const supabaseUrl = Deno.env.get('SUPABASE_URL')!;
  const anonKey     = Deno.env.get('SUPABASE_ANON_KEY')!;
  const groqKey     = Deno.env.get('GROQ_API_KEY');

  if (!groqKey) return json(500, { error: 'groq_key_not_configured' });

  const client = createClient(supabaseUrl, anonKey, {
    global: { headers: { Authorization: auth } },
  });

  let articles: { id: string; title: string; body: string; category: string }[] = [];

  try {
```

```
  // @ts-ignore - Supabase.ai injected by edge runtime
  const session = new Supabase.ai.Session('gte-small');
  const output = await session.run(prompt, { mean_pool: true, normalize: true });
  const embedding = Array.from(output as Float32Array);

  const { data } = await client.rpc('match_articles', {
    query_embedding: embedding,
    match_count: 3,
    p_age_months: childAgeMonths,
  });
  articles = (data ?? []) as typeof articles;
} catch {
  // Embeddings not yet computed - fall through to text fallback.
}

if (articles.length === 0) {
  const keyword = prompt.split(/\s+/).slice(0, 4).join(' ');
  const { data } = await client
    .from('articles')
    .select('id, title, body, category')
    .or(`title.ilike.%${keyword}%,body.ilike.%${keyword}%`)
    .limit(3);
  articles = (data ?? []) as typeof articles;
}

const contextBlock = articles.length > 0
  ? articles.map((a) => `## ${a.title}\n\n${a.body}`).join('\n\n---\n\n')
  : null;

const promptLang = /[-]/i.test(prompt) ? 'uk' : 'en';

const systemMessage = [
  'You are a helpful assistant for parents of newborns. Be concise, practical, warm.',
  childAgeMonths !== null ? `Child age: ${childAgeMonths} months.` : null,
  contextBlock
    ? `Use the following articles as context:\n\n${contextBlock}`
    : 'Answer from your general knowledge.',
  'If the question is unrelated to child health or development, politely explain.',
  promptLang === 'uk'
    ? 'LANGUAGE: Ukrainian only.'
    : 'LANGUAGE: English only.',
].filter(Boolean).join('\n\n');

const groqRes = await fetch('https://api.groq.com/openai/v1/chat/completions', {
  method: 'POST',
  headers: {
```

```
        'Authorization': `Bearer ${groqKey}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        model: 'llama-3.3-70b-versatile',
        messages: [
          { role: 'system', content: systemMessage },
          { role: 'user',   content: prompt },
        ],
        max_tokens: 600,
      }),
    });

    if (!groqRes.ok) {
      const detail = await groqRes.text();
      return json(502, { error: 'groq_failed', detail });
    }

    const groqData = await groqRes.json();
    const answer: string = groqData.choices?.[0]?.message?.content ?? '';

    const sources: ArticleSource[] = articles.map((a) => ({
      id: a.id,
      title: a.title,
      category: a.category,
    }));

    return json(200, { answer, sources });
  });
```